

AWG Benchmark Report

Multi-SWE-bench Go pilot - enhanced PDF report with test counts, charts, and external benchmark context

Executive verdict

AWG gives a positive result in this pilot. The cleanest evidence is Mode C, where the same model, checkout, issue text, budget, and test command are held constant, and only context/tools differ. Mode C improved the average judged score from 86.7 in the plain tool-assisted mode to 92.2, and improved full test-pass tasks from 5/10 to 7/10.

The strongest honest claim: AWG measurably improves an already-capable coding agent by adding project-specific architectural context and governance pressure. It is not yet proof that AWG universally solves software engineering. The pilot is small: 10 Go tasks from one repository.

Headline scorecard

Mode	Primary runs	Scored runs	Avg score	Zero-filled score	Full test-pass tasks	Tests passed	Build pass	Avg cost	Avg turns
A - agent alone	10	10	71.7	71.7	3/10	88/144	9/10	\$2.15	36.0
B - agent + tools	10	10	86.7	86.7	5/10	127/144	10/10	\$1.43	25.4
C - agent + AWG	10	10	92.2	92.2	7/10	135/144	10/10	\$1.54	25.6
D - contract-first	9	5	94.8	52.7	4/9	62/130	9/9	\$1.13	15.2

Important Mode D caveat: Mode D completed runs scored very high, but 4 of 9 primary Mode D records had no final judged score. The zero-filled score is therefore the more conservative readiness signal for Mode D today.

What was actually tested?

The harness runs Go issue-resolution tasks from Multi-SWE-bench against GitHub CLI. It evaluates three baseline modes plus an optional contract-first lane: A = agent alone, B = agent plus normal tools, C = agent plus AWG, D = agent plus frozen contracts. The fairness rule keeps the model, checkout, issue text, budget, and test command fixed; only context and tools change. The task runner records checkout, agent patch, verification, and scoring artifacts.

The scoring split is 100 points: 40 mechanical test-pass points from f2p/p2p verification, plus 60 judged points for localization, minimality, architecture/rule safety, test/evidence quality, and explanation. This matters because the report does not rely only on prose judgement; the test lane is a separate mechanical signal.

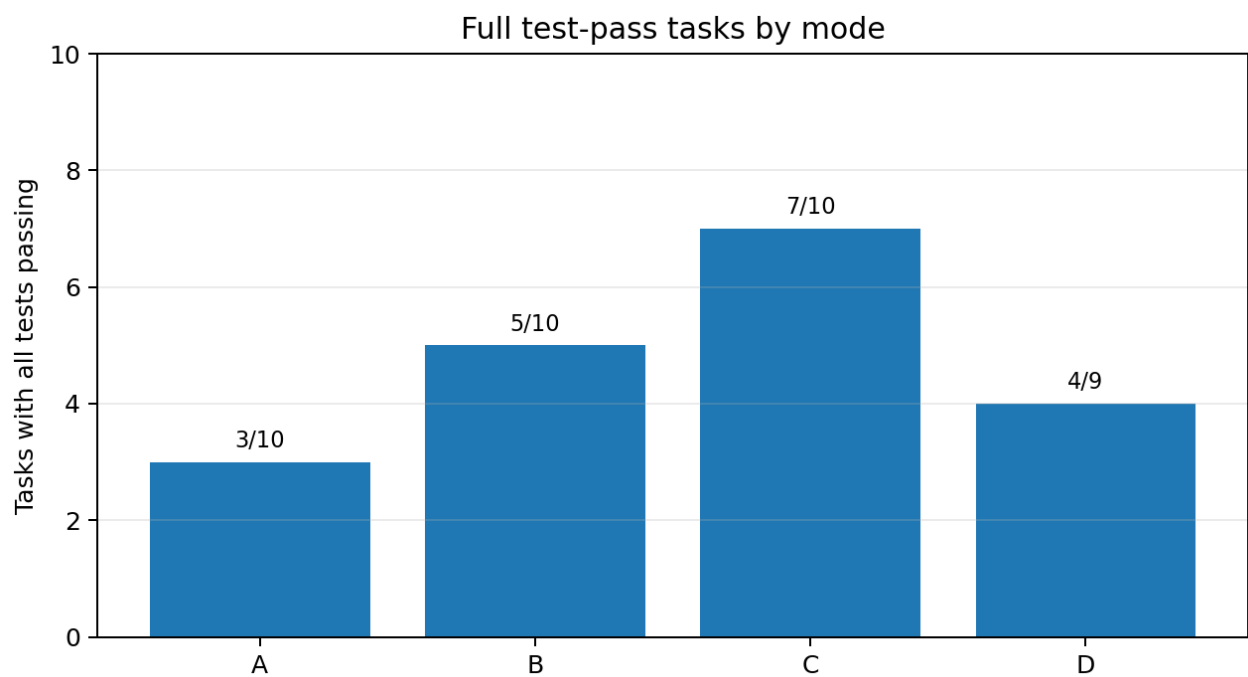
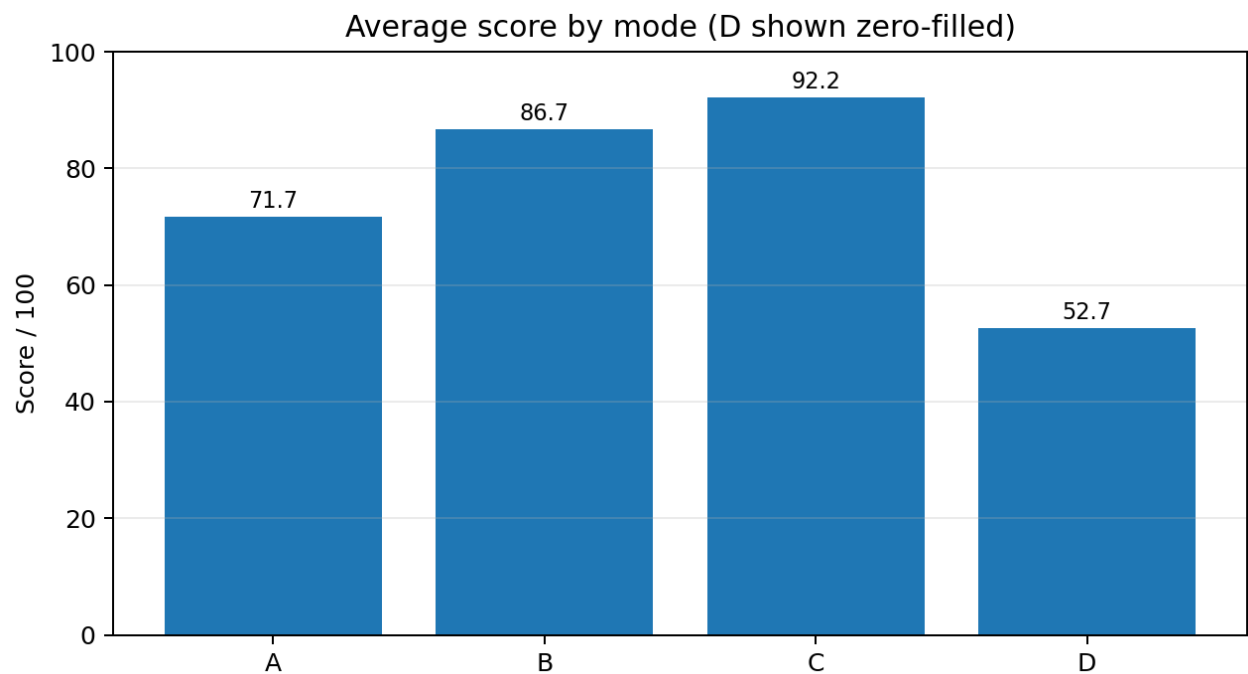
Pilot size and test volume

Item	Count / meaning
Unique tasks	10 Go tasks from github.com/cli/cli
Primary run records	39 total: A=10, B=10, C=10, D=9
Archived retry/attempt records	15 total: C=7, D=8
A/B/C mechanical test cases	144 test-case executions per mode across the 10 tasks
Mode C test cases passed	135/144, versus 127/144 in Mode B and 88/144 in Mode A
Full task pass definition	tests_passed == tests_run for that task/mode
Trusted verifier path	Multi-SWE-bench Docker harness when MSWEB_HARNESS=1; local go build/test is iteration path

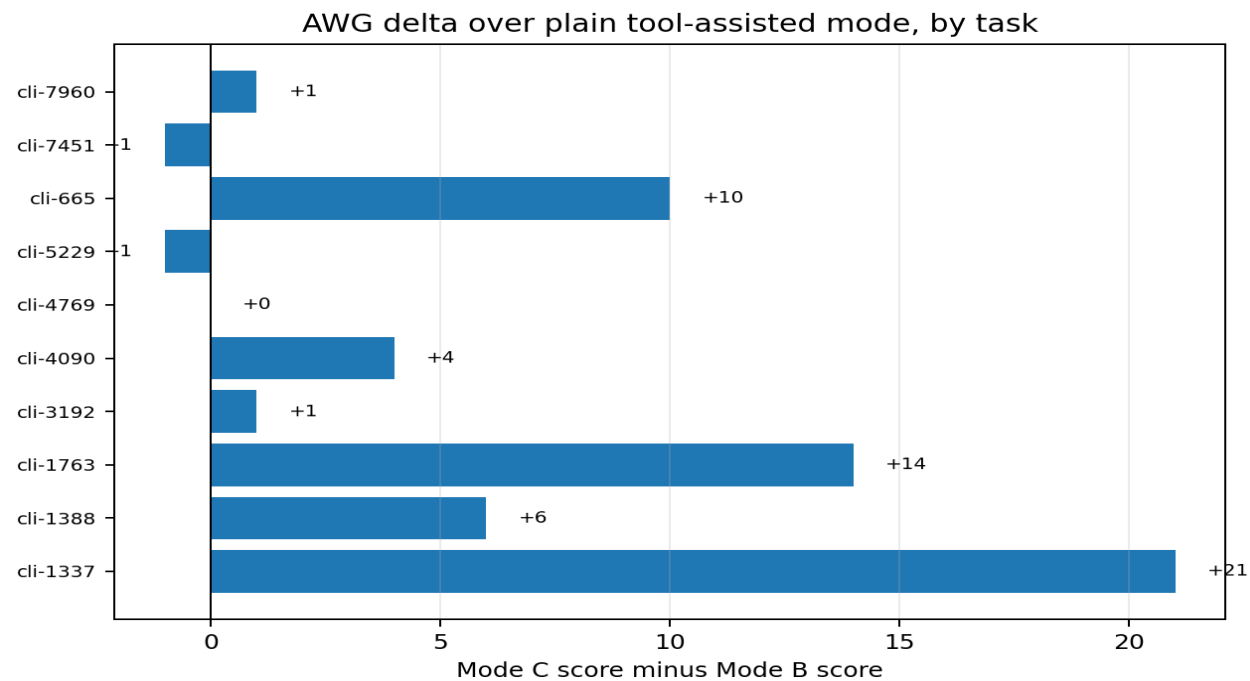
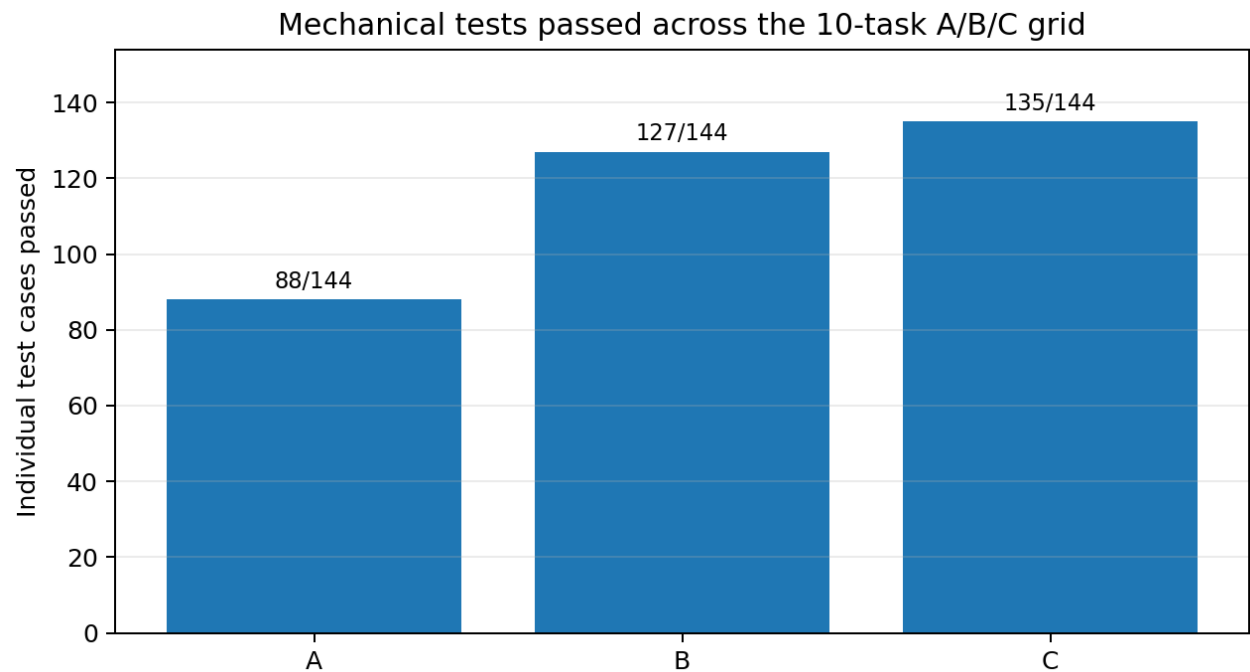
Task-level test counts

Task	Tests in suite	A passed	B passed	C passed	C result	C-B score delta
cli__cli-1337	16	10/16	10/16	16/16	pass	+21
cli__cli-1388	15	2/15	15/15	15/15	pass	+6
cli__cli-1763	10	10/10	8/10	10/10	pass	+14
cli__cli-3192	12	12/12	12/12	12/12	pass	+1
cli__cli-4090	16	14/16	14/16	14/16	partial/fail	+4
cli__cli-4769	10	0/10	10/10	10/10	pass	+0
cli__cli-5229	18	0/18	18/18	18/18	pass	-1
cli__cli-665	16	16/16	16/16	16/16	pass	+10
cli__cli-7451	17	14/17	14/17	14/17	partial/fail	-1
cli__cli-7960	14	10/14	10/14	10/14	partial/fail	+1

Charts: AWG signal



Mechanical test signal



The C-over-B chart is the most important product chart: Mode C beats the plain tool-assisted baseline on 7 of 10 tasks, ties/near-ties on one, and loses narrowly on two. That is the evidence that AWG adds value beyond just giving the agent tools.

Paired comparisons

Comparison	Shared tasks	Avg score delta	Wins	Losses	Ties	Avg tests-passed de
C vs A	10	20.5	9	0	1	4.7
C vs B	10	5.5	7	2	1	0.8
B vs A	10	15.0	6	3	1	3.9
D vs C scored only	5	-1.8	2	3	0	-1.4
D vs C zero-filled	9	-40.7	2	7	0	-7.0

Interpretation: B already improves strongly over A. That is expected: a modern coding agent with tool access beats an isolated model prompt. AWG Mode C then improves over B by another +5.5 points and +0.8 tests passed per task on average. The positive signal is therefore not simply "tools help". It is "project-aware governance/context helps above a tool baseline."

What does Mode C add?

- Briefing: project-specific context around the files and task.
- Impact/resolve: guidance about related files, symbols, and architectural blast radius.
- Preflight: pressure against drifting outside the intended contract or authority boundary.
- Result in the pilot: fewer wandering edits, stronger localization, and more full test-pass outcomes than Mode B.

Mode D diagnosis

Mode D is interesting but not yet ready as the headline. Completed Mode D runs averaged 94.8, but only 5/9 primary D records received a judged score and 4/9 had no final score. The conservative zero-filled score is 52.7. Treat D as a prototype lane with a high ceiling and harness/finalization risk, not as the current product proof.

External context: how "plain tools" usually behave

The pilot already contains the best direct baseline: Mode B, same model plus normal tools but without AWG. That is the apples-to-apples comparison. External benchmarks are not directly comparable because they use different repositories, scaffolds, task sets, models, budgets, and scoring, but they help interpret whether this effect size is meaningful.

SWE-bench: Real GitHub issue-resolution benchmark; original SWE-bench used 2,294 tasks from 12 Python repositories. This confirms the benchmark family is serious issue repair, not demo frosting.

Multi-SWE-bench: Multilingual issue-resolution benchmark; reported as 1,632 high-quality instances across 7 languages. The AWG pilot uses the Go subset and therefore tests real repo repair patterns.

SWE-Skills-Bench 2026: Generic skill injection showed limited average gain (+1.2%) and many zero-gain skills. AWG therefore needs measured uplift over tools; Mode C provides that inside this pilot.

Claw-SWE-Bench 2026: Harness and adapter design can dominate results; one reported same-backbone contrast was 19.1% versus 73.4%. This is why the report focuses on the fixed A/B/C comparison, not unrelated leaderboards.

Conclusion from context: AWG is not proving that context always helps. It is proving that this specific architecture-aware context layer produced a measurable gain over a plain tool-assisted run under fixed conditions. That is a valuable product signal.

Limits and confidence

- Sample size is small: 10 tasks from one Go repository. Strong signal, not universal proof.
- Scores include judged dimensions. The mechanical test lane is separate, but 60/100 points still depend on an LLM judge.
- Mode D should be described as experimental until no-score finalization failures are eliminated.
- The result proves the Mode C bundle, not which individual AWG feature caused most of the gain.
- Human mergeability is inferred from tests and judged localization, not from real maintainer acceptance.

Confidence rating: medium-high for "AWG Mode C helps on this benchmark slice"; medium for "AWG generalizes across repos"; low for "Mode D is production-ready".

Recommended next benchmark

- Run 25-50 tasks across at least 3 repositories, still keeping A/B/C fixed and comparable.
- Tag tasks by type: single-file, multi-file, API contract, cross-component, test-only, behavior regression.
- Keep Mode D separate until the wrapper never replaces a good patch with a no-score final.
- Add ablations: briefing-only, impact-only, preflight-only, full Mode C.
- Report no-score failure rate as a first-class metric, not only average score.
- Preserve the exact test-count table for every run. No more ghost statistics.

Appendix A - Full task table

Task	A score / tests	B score / tests	C score / tests	D score / tests	C-B	C-A	C files +/-
cli__cli-1337	71 / 10/16	72 / 10/16	93 / 16/16	100 / 16/16	+21	+22	3; +14/-14
cli__cli-1388	63 / 2/15	93 / 15/15	99 / 15/15	97 / 15/15	+6	+36	1; +67/-20
cli__cli-1763	93 / 10/10	85 / 8/10	99 / 10/10	85 / 3/10	+14	+6	2; +83/-44
cli__cli-3192	85 / 12/12	96 / 12/12	97 / 12/12	n/a / 0/12	+1	+12	7; +126/-10
cli__cli-4090	78 / 14/16	78 / 14/16	82 / 14/16	n/a / 0/16	+4	+4	2; +13/-8
cli__cli-4769	55 / 0/10	97 / 10/10	97 / 10/10	100 / 10/10	+0	+42	1; +11/-12
cli__cli-5229	20 / 0/18	96 / 18/18	95 / 18/18	92 / 18/18	-1	+75	3; +25/-14
cli__cli-665	92 / 16/16	89 / 16/16	99 / 16/16	n/a / 0/16	+10	+7	2; +24/-19
cli__cli-7451	78 / 14/17	80 / 14/17	79 / 14/17	n/a / 0/17	-1	+1	4; +21/-16
cli__cli-7960	82 / 10/14	81 / 10/14	82 / 10/14	-	+1	+0	2; +15/-5

Appendix B - Source notes

Primary data source: /mnt/data/records.tar.gz, analyzed from primary YAML records under records/. Archived *-attempt0.yaml files were treated as retry history, not headline final results. The v1-empty-context-backup directory was excluded from the headline numbers.

Harness source: uploaded README.md. It defines the A/B/C/D modes, fairness rule, pipeline, verification backends, and rubric split.

External references used for context, not direct numeric comparison:

- Multi-SWE-bench GitHub project and dataset description.
- SWE-bench paper: "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?"
- SWE-Skills-Bench 2026 paper on the limited average value of generic agent skills.
- Claw-SWE-Bench 2026 paper on the importance of harness/adaptor design.

Report generated: 2026-07-04 23:11:23 local sandbox time.